UNIVERSIDAD DE LOS LLANOS
INGENIERÍA DE SISTEMAS
OPTIMIZACIÓN

TÉCNICAS METAHEURÍSTICAS – ALGORITMO GENÉTICO
PRÁCTICA

Técnicas Heurísticas y Metaheurísticas en Optimización

Las técnicas heurísticas y metaheurísticas son métodos utilizados para resolver problemas de optimización complejos donde los métodos exactos tradicionales resultan muy costosos o imposibles de aplicar en tiempo razonable.

Son ampliamente utilizadas en:

- Ingeniería,
- Inteligencia Artificial,
- Investigación de Operaciones,
- Logística,
- Ciencia de Datos,
- Redes,
- Finanzas,
- Planeación y producción.

1. ¿Qué son las técnicas heurísticas?

Una **heurística** es una estrategia práctica que busca encontrar soluciones buenas o aceptables de manera rápida, aunque no garantice la solución óptima global.

La palabra heurística proviene del griego:
"Heuriskein" = descubrir

Características de las heurísticas

- Buscan soluciones rápidas.
- Reducen tiempo computacional.
- Son aproximadas.
- No garantizan optimalidad.
- Se diseñan específicamente para un problema.

Ejemplos de heurísticas

- Método del vecino más cercano.

- Regla FIFO.
- Algoritmos voraces (*greedy*).
- Búsqueda local.

Ejemplo sencillo

Problema:
Encontrar la ruta más corta para repartir productos.

Heurística:
“Siempre visitar la ciudad más cercana”.

Puede no ser la mejor ruta global, pero suele producir soluciones razonables rápidamente.

2. ¿Qué son las metaheurísticas?

Las **metaheurísticas** son estrategias generales de alto nivel diseñadas para explorar inteligentemente el espacio de soluciones y evitar quedar atrapadas en óptimos locales.

Son más sofisticadas que las heurísticas simples.

Características de las metaheurísticas

- Son métodos generales.
- Aplicables a muchos problemas.
- Buscan aproximarse al óptimo global.
- Incorporan aleatoriedad.
- Balancean:
    - exploración,
    - explotación.

Exploración y explotación

Exploración

Buscar nuevas regiones del espacio de soluciones.

Explotación

Mejorar soluciones ya encontradas.

Clasificación de las Metaheurísticas

Según el número de soluciones manejadas

A. Metaheurísticas basadas en trayectoria

Trabajan con una sola solución.

Ejemplos:

- Recocido Simulado (*Simulated Annealing*)
- Búsqueda Tabú (*Tabu Search*)
- Descenso de Gradiente

B. Metaheurísticas poblacionales

Trabajan con múltiples soluciones simultáneamente.

Ejemplos:

- Algoritmos Genéticos
- Optimización por Enjambre de Partículas
- Colonias de Hormigas
- Evolución Diferencial

Según la inspiración natural

Tipo	**Inspiración**
Evolutivas	Evolución biológica
Inteligencia colectiva	Colonias o enjambres
Físicas	Procesos físicos
Sociales	Comportamiento humano

Ejemplos importantes de metaheurísticas

Metaheurística	**Inspiración**
Algoritmos Genéticos	Evolución natural
PSO	Movimiento de enjambres
ACO	Colonias de hormigas
Simulated Annealing	Enfriamiento de metales
Búsqueda Tabú	Memoria adaptativa

Utilidad de heurísticas y metaheurísticas

Son útiles cuando:

- el problema es NP-hard,
- existen muchísimas combinaciones,
- los métodos exactos son demasiado lentos.

Aplicaciones

Ingeniería

- Diseño de redes
- Optimización estructural

Logística

- Rutas de vehículos
- Distribución

Inteligencia Artificial

- Entrenamiento
- Ajuste de parámetros

Producción

- Scheduling
- Planeación

Finanzas

- Portafolios de inversión

3. ¿Qué son los Algoritmos Genéticos?

Los **Algoritmos Genéticos (GA)** son metaheurísticas inspiradas en el proceso de evolución biológica y selección natural propuesto por:

Charles Darwin

Idea principal

Las mejores soluciones sobreviven y se combinan para generar nuevas soluciones potencialmente mejores.

Analogía biológica

Biología	**Algoritmos Genéticos**
Individuo	Solución
Cromosoma	Representación de solución
Gen	Variable
Población	Conjunto de soluciones
Evolución	Mejora iterativa
Selección natural	Escoger mejores soluciones

Conceptos clave de los Algoritmos Genéticos

1. Población

Conjunto de soluciones candidatas.

Ejemplo:
P1 = 101010
P2 = 111000
P3 = 010111

2. Cromosoma

Representación codificada de una solución.

Puede ser:

- binaria,
- entera,
- real.

3. Gen

Cada componente individual del cromosoma.

Ejemplo:
101010

Cada bit es un gen.

4. Función de aptitud (Fitness)

Evalúa qué tan buena es una solución.

Es el corazón del algoritmo.

Ejemplo:
f(x) = beneficio total

5. Selección

Escoge los mejores individuos para reproducirse.

Métodos:

- Ruleta
- Torneo
- Ranking

---

6. Cruce (Crossover)

Combina información genética de dos padres.

Ejemplo:
Padre 1: 111000
Padre 2: 000111

Hijo: 111111

7. Mutación

Introduce cambios aleatorios.

Evita convergencia prematura.

Ejemplo:
Antes: 101010
Después:101110

8. Reemplazo

Se decide qué individuos permanecen en la población.

Procedimiento general de un Algoritmo Genético

Paso 1. Definir el problema

Ejemplo:
Maximizar ganancias.

Paso 2. Codificar soluciones

Representar variables como cromosomas.

Ejemplo:
101101

Paso 3. Generar población inicial

Crear soluciones aleatorias.

Paso 4. Evaluar fitness

Calcular calidad de cada individuo.

Paso 5. Seleccionar padres

Elegir mejores soluciones.

Paso 6. Aplicar cruce

Combinar padres.

Paso 7. Aplicar mutación

Introducir diversidad.

Paso 8. Crear nueva población

Generar siguiente generación.

Paso 9. Verificar criterio de parada

Ejemplo:

- número de generaciones,
- convergencia,
- tiempo máximo.

Diagrama conceptual del proceso

Población inicial
↓
Evaluación fitness
↓
Selección
↓
Cruce
↓
Mutación
↓
Nueva población
↓
¿Criterio de parada?
Sí → Mejor solución
No → Repetir

Ventajas de los Algoritmos Genéticos

Ventaja	**Descripción**
Flexibles	Aplicables a muchos problemas
Globales	Exploran gran espacio
Robustez	Funcionan en problemas complejos
No requieren derivadas	Útiles en funciones discontinuas

Desventajas

Desventaja	**Descripción**
Alto costo computacional	Muchas evaluaciones
No garantizan óptimo exacto	Aproximados
Sensibles a parámetros	Tamaño población, mutación, etc.

Aplicaciones reales de Algoritmos Genéticos

- Diseño de rutas
- Redes neuronales
- Optimización industrial
- Robótica
- Planeación energética
- Diseño aeronáutico
- Machine Learning
- Bioinformática

PRÁCTICA:

1. Elaborar un mapa mental con las Técnicas Heurísticas y Metaheurísticas de Optimización, con su explicación.
2. Implementación de Algoritmo Genético:

Ejercicio básico de Optimización de Rutas con Algoritmo Genético
Contexto del problema
Una empresa de transporte necesita encontrar la mejor ruta para repartir productos desde una bodega central hacia varias ciudades, recorriendo la menor distancia posible y regresando al punto inicial.
Este problema corresponde a una versión simplificada del:
Problema del Viajante (TSP — Traveling Salesman Problem)

Ciudades del ejercicio
La empresa debe visitar las siguientes ciudades:

Ciudad	**Código**
Bogotá	A
Villavicencio	B
Granada	C
Acacías	D

Matriz de distancias (km)

	A	**B**	**C**	**D**
A	0	30	50	40
B	30	0	25	35
C	50	25	0	15
D	40	35	15	0

Objetivo
Encontrar la ruta con menor distancia total usando un:
Algoritmo Genético

Conceptos fundamentales del Algoritmo Genético aplicados al ejercicio

1. Cromosoma
Representa una solución posible.
En este problema:

- un cromosoma es un orden de ciudades.

Ejemplo:
A - B - C - D - A
Interpretación:

- sale de A,
- visita B,
- luego C,
- luego D,
- regresa a A.

2. Gen
Cada ciudad representa un gen.
Ejemplo:
[A] [B] [C] [D]

3. Población
Conjunto de rutas posibles.
Ejemplo:

Individuo	**Ruta**
P1	A-B-C-D-A
P2	A-C-B-D-A
P3	A-D-C-B-A
P4	A-B-D-C-A

4. Fitness (aptitud)
Mide qué tan buena es la solución.
En este caso:
Menor distancia = mayor fitness

Paso 1. Generar población inicial
Supongamos:

Individuo	**Ruta**
P1	A-B-C-D-A
P2	A-C-B-D-A
P3	A-D-C-B-A
P4	A-B-D-C-A

Paso 2. Evaluar fitness
Calculamos distancia total.

Individuo P1
Ruta:
A → B → C → D → A
Distancia:

- A-B = 30
- B-C = 25
- C-D = 15
- D-A = 40

Total:
30 + 25 + 15 + 40 = 110 km

Individuo P2
A → C → B → D → A
Distancia:

- A-C = 50
- C-B = 25
- B-D = 35
- D-A = 40

Total:
150 km

Individuo P3
A → D → C → B → A
Distancia:

- A-D = 40
- D-C = 15
- C-B = 25
- B-A = 30

Total:
110 km

Individuo P4
A → B → D → C → A
Distancia:

- A-B = 30
- B-D = 35
- D-C = 15
- C-A = 50

Total:
130 km

Resumen Fitness

Individuo	Distancia
P1	110
P2	150
P3	110
P4	130

Las mejores soluciones son:

- P1
- P3

Paso 3. Selección
Se eligen los mejores individuos para reproducirse.
Seleccionamos:

- P1
- P3

Concepto clave: Selección natural
Las mejores soluciones tienen mayor probabilidad de sobrevivir y generar descendencia.
Inspirado en:
Charles Darwin

Paso 4. Cruce (Crossover)
Combinar información de dos padres.

Padres seleccionados
Padre 1
A-B-C-D-A
Padre 2
A-D-C-B-A

Cruce de un punto
Supongamos corte después del segundo gen.

Hijo generado
Tomamos:

- inicio del Padre 1,
- resto válido del Padre 2.

Resultado:
A-B-D-C-A

Concepto clave: Herencia genética
El hijo hereda características de ambos padres.

Paso 5. Mutación
Modificar aleatoriamente un gen.

Antes
A-B-D-C-A

Después de mutación
Intercambiamos D y C:
A-B-C-D-A

Concepto clave: Diversidad genética

La mutación:

- evita soluciones repetidas,
- ayuda a explorar nuevas posibilidades,
- reduce riesgo de óptimos locales.

Paso 6. Nueva generación

La nueva población podría ser:

Individuo	Ruta	Distancia
H1	A-B-C-D-A	110
H2	A-D-C-B-A	110
H3	A-B-D-C-A	130
H4	A-C-B-D-A	150

Paso 7. Criterio de parada

El algoritmo termina cuando:

- se alcanza número máximo de generaciones,
- no hay mejoras,
- o se encuentra solución suficientemente buena.

Mejor solución encontrada

A-B-C-D-A
Distancia:
110 km

Explicación de los conceptos fundamentales

Concepto	Explicación
Población	Conjunto de rutas
Cromosoma	Ruta completa
Gen	Ciudad
Fitness	Calidad de la ruta
Selección	Elegir mejores rutas
Cruce	Combinar rutas
Mutación	Cambios aleatorios
Generación	Nueva población

Ventajas del Algoritmo Genético en rutas

- Explora muchas soluciones.
- Funciona bien en problemas complejos.
- Flexible.
- Útil cuando existen muchísimas combinaciones.

Limitaciones

- No garantiza óptimo exacto.
- Puede requerir muchas iteraciones.
- Sensible a parámetros:

- mutación,
- población,
- cruce.

A continuación se presenta un código en Python que implementa el problema de optimización de rutas usando un **Algoritmo Genético**, evidenciando claramente sus elementos fundamentales:

- Población
- Cromosoma
- Fitness
- Selección
- Cruce
- Mutación
- Evolución de generaciones

Código en Python — Algoritmo Genético para Optimización de Rutas

```
import random

# ========================================
# MATRIZ DE DISTANCIAS
# ========================================

distancias = {
    ('A', 'B'): 30,
    ('A', 'C'): 50,
    ('A', 'D'): 40,
    ('B', 'C'): 25,
    ('B', 'D'): 35,
    ('C', 'D'): 15
}

# Función para obtener distancia entre ciudades
def obtener_distancia(ciudad1, ciudad2):
    if ciudad1 == ciudad2:
        return 0

    if (ciudad1, ciudad2) in distancias:
        return distancias[(ciudad1, ciudad2)]

    return distancias[(ciudad2, ciudad1)]


# ========================================
# CONFIGURACIÓN DEL ALGORITMO GENÉTICO
# ========================================

ciudades = ['B', 'C', 'D']
```

```
TAM_POBLACION = 6
GENERACIONES = 10
TASA_MUTACION = 0.2


# ========================================
# 1. CREAR CROMOSOMA
# ========================================

def crear_individuo():
    ruta = ciudades[:]
    random.shuffle(ruta)
    return ruta


# ========================================
# 2. CREAR POBLACIÓN INICIAL
# ========================================

def crear_poblacion():
    poblacion = []

    for _ in range(TAM_POBLACION):
        poblacion.append(crear_individuo())

    return poblacion


# ========================================
# 3. FUNCIÓN FITNESS
# ========================================

def calcular_distancia(ruta):

    ruta_completa = ['A'] + ruta + ['A']

    distancia_total = 0

    for i in range(len(ruta_completa) - 1):
        distancia_total += obtener_distancia(
            ruta_completa[i],
            ruta_completa[i + 1]
        )

    return distancia_total


# ========================================
```

```
# 4. SELECCIÓN
# ========================================

def seleccionar_padres(poblacion):

    poblacion_ordenada = sorted(
        poblacion,
        key=lambda x: calcular_distancia(x)
    )

    return poblacion_ordenada[:2]


# ========================================
# 5. CRUCE (CROSSOVER)
# ========================================

def cruce(padre1, padre2):

    punto = random.randint(1, len(padre1) - 1)

    hijo = padre1[:punto]

    for ciudad in padre2:
        if ciudad not in hijo:
            hijo.append(ciudad)

    return hijo


# ========================================
# 6. MUTACIÓN
# ========================================

def mutacion(individuo):

    if random.random() < TASA_MUTACION:

        i = random.randint(0, len(individuo) - 1)
        j = random.randint(0, len(individuo) - 1)

        individuo[i], individuo[j] = individuo[j], individuo[i]

    return individuo


# ========================================
# 7. EVOLUCIÓN
```

```
# ========================================

def algoritmo_genetico():

    poblacion = crear_poblacion()

    print("\nPOBLACIÓN INICIAL\n")

    for individuo in poblacion:
        print(
            ['A'] + individuo + ['A'],
            "Distancia:",
            calcular_distancia(individuo)
        )

    # Evolución
    for generacion in range(GENERACIONES):

        print("\n==============================")
        print(f"GENERACIÓN {generacion + 1}")
        print("==============================")

        # Selección
        padre1, padre2 = seleccionar_padres(poblacion)

        print("\nMEJORES PADRES:")
        print("Padre 1:", ['A'] + padre1 + ['A'])
        print("Padre 2:", ['A'] + padre2 + ['A'])

        nueva_poblacion = []

        # Crear nueva generación
        while len(nueva_poblacion) < TAM_POBLACION:

            hijo = cruce(padre1, padre2)

            hijo = mutacion(hijo)

            nueva_poblacion.append(hijo)

        poblacion = nueva_poblacion

        # Mostrar generación
        for individuo in poblacion:
            print(
                ['A'] + individuo + ['A'],
                "Distancia:",
                calcular_distancia(individuo)
```

```
        )

    # Mejor solución final
    mejor = min(
        poblacion,
        key=lambda x: calcular_distancia(x)
    )

    print("\n==================================")
    print("MEJOR RUTA ENCONTRADA")
    print("==================================")

    print(
        "Ruta:",
        ['A'] + mejor + ['A']
    )

    print(
        "Distancia total:",
        calcular_distancia(mejor),
        "km"
    )


# ========================================
# EJECUCIÓN
# ========================================

algoritmo_genetico()
```

Explicación de los elementos del Algoritmo Genético en el código

1. Cromosoma
Representa una ruta:
['B', 'C', 'D']
La ruta completa será:
A → B → C → D → A

2. Población
Conjunto de soluciones iniciales:
crear_poblacion()

3. Fitness
La calidad de la solución se mide con:
calcular_distancia()
Menor distancia = mejor fitness.

4. Selección

Escoge los mejores individuos:
seleccionar_padres()

5. Cruce
Combina padres:
cruce()
Genera hijos heredando características.

6. Mutación
Introduce diversidad:
mutacion()
Intercambia ciudades aleatoriamente.

7. Evolución
Proceso iterativo:
for generacion in range(GENERACIONES)
Cada generación mejora soluciones.

Posible salida del programa
POBLACIÓN INICIAL

['A', 'B', 'D', 'C', 'A'] Distancia: 130
['A', 'C', 'D', 'B', 'A'] Distancia: 130
['A', 'B', 'C', 'D', 'A'] Distancia: 110

...

MEJOR RUTA ENCONTRADA

Ruta: ['A', 'B', 'C', 'D', 'A']
Distancia total: 110 km

Conceptos académicos evidenciados

Concepto	**Evidencia en el código**
Individuo	Ruta
Cromosoma	Lista de ciudades
Gen	Ciudad
Población	Conjunto de rutas
Fitness	Distancia total
Selección	Mejores padres
Cruce	Combinación de padres
Mutación	Intercambio aleatorio
Evolución	Iteraciones generacionales